Matplotlib Foundations

2.1 Getting Started

2.1.1 What is Matplotlib?

Matplotlib is a Python library used to create data visualizations.

It can create:

- Line charts
- Bar charts
- Histograms
- Scatter plots
- Pie charts
- Box plots
- Subplots
- Customized publication/report-quality figures

The most commonly used part of Matplotlib is:

```
import matplotlib.pyplot as plt
```

Here:

- `matplotlib` → the library
- `pyplot` → plotting interface
- `plt` → commonly used alias

2.1.2 Importing Matplotlib

The standard import is:

```
import matplotlib.pyplot as plt
```

Then we can create a simple plot:

```
plt.plot([1, 2, 3, 4, 5])
plt.show()
```

When only one list is supplied, Matplotlib automatically uses the positions:

```
x → 0, 1, 2, 3, 4
y → 1, 2, 3, 4, 5
```

So this:

```
plt.plot([10, 20, 30, 40])
```

is effectively plotting:

```
x = [0, 1, 2, 3]
y = [10, 20, 30, 40]
```

2.1.3 `pyplot`

`pyplot` provides functions for creating and customizing plots.

Some important functions:

```
plt.plot()
plt.bar()
plt.scatter()
plt.hist()
plt.pie()
plt.boxplot()
plt.title()
plt.xlabel()
plt.ylabel()
plt.legend()
plt.grid()
plt.show()
```

Think of `plt` as the main interface you'll use to tell Matplotlib:

> "Create this chart and customize it this way."

2.1.4 Basic `plot()`

The basic syntax is:

```
plt.plot(x, y)
plt.show()
```

Example:

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [120, 150, 170, 160, 200]

plt.plot(months, sales)
plt.show()
```

Here:

- `months` → x-axis
- `sales` → y-axis

Adding a title

```
plt.plot(months, sales)
plt.title("Monthly Sales")
plt.show()
```

Adding axis labels

```
plt.plot(months, sales)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```

2.1.5 Figure and Axes

This is one of the **most important Matplotlib concepts** to understand.

A visualization generally has two important objects:

Figure

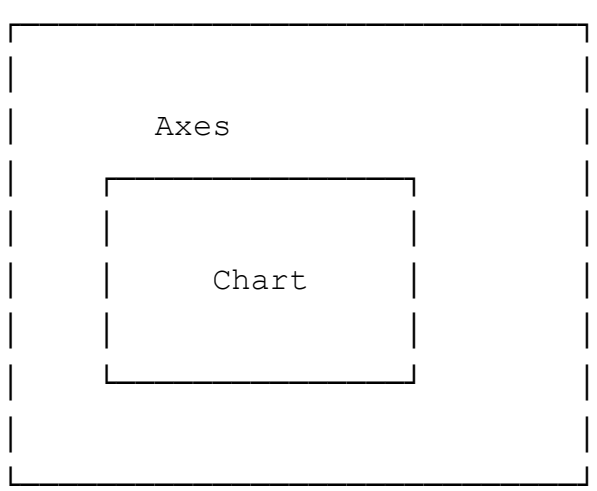
The **Figure** is the overall canvas.

Think of it as the entire piece of paper containing your visualization.

Axes

An **Axes** is the actual plotting area where the data is displayed.

A single Figure can contain:



Importantly, **Axes does not mean only the x-axis or y-axis**. In Matplotlib terminology, an `Axes` object represents the whole plotting area.

Object-oriented approach

You can explicitly create a Figure and Axes:

```
fig, ax = plt.subplots()

ax.plot(months, sales)

ax.set_title("Monthly Sales")
ax.set_xlabel("Month")
ax.set_ylabel("Sales")

plt.show()
```

This approach becomes especially important when creating **multiple charts**.

2.1.6 `plt.show()`

`plt.show()` displays the figure.

Example:

```
plt.plot([1, 2, 3, 4])
plt.show()
```

In scripts, it's good practice to explicitly call:

```
plt.show()
```

In Jupyter notebooks, plots can sometimes appear automatically, but students should still understand what `show()` does.

2.1.7 Understanding the Plotting Workflow

Teach students to think about visualization as a sequence:

```
1. Import library
        ↓
2. Prepare data
        ↓
3. Create Figure/Axes
        ↓
4. Create chart
        ↓
5. Customize chart
        ↓
6. Add labels/title/legend
        ↓
7. Display or save
```

Example:

```
import matplotlib.pyplot as plt
```

```
months = ["Jan", "Feb", "Mar", "Apr"]
sales = [100, 140, 120, 180]

fig, ax = plt.subplots(figsize=(8, 5))

ax.plot(months, sales)

ax.set_title("Monthly Sales")
ax.set_xlabel("Month")
ax.set_ylabel("Sales")

ax.grid(True)

plt.show()
```

This workflow should become second nature.

2.2 Basic Customization

Creating a chart is only the first step.

A good analyst should make the visualization:

- understandable
- readable
- appropriately labeled
- visually consistent
- easy to interpret

2.2.1 Figure Size

Use:

```
plt.figure(figsize=(10, 6))
```

or:

```
fig, ax = plt.subplots(figsize=(10, 6))
```

The values represent:

```
(width, height)
```

in inches.

Example:

```
plt.figure(figsize=(12, 5))
plt.plot(months, sales)
plt.show()
```

Why does size matter?

A chart that's too small can make:

- labels difficult to read
- data points difficult to distinguish
- legends overlap

2.2.2 Chart Title

```
plt.title("Monthly Sales")
```

Object-oriented version:

```
ax.set_title("Monthly Sales")
```

You can customize the title:

```
plt.title(
    "Monthly Sales",
    fontsize=18,
    fontweight="bold"
)
```

2.2.3 X-axis and Y-axis Labels

```
plt.xlabel("Month")
plt.ylabel("Sales")
```

Example:

```
plt.plot(months, sales)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales ($)")

plt.show()
```

A good chart should make it obvious what each axis represents.

2.2.4 Axis Limits

You can control the visible range.

X-axis

```
plt.xlim(0, 10)
```

Y-axis

```
plt.ylim(0, 250)
```

Object-oriented:

```
ax.set_xlim(0, 10)
ax.set_ylim(0, 250)
```

Why use limits?

To:

- focus on a particular range
- improve readability
- prevent excessive empty space

But students should be careful not to manipulate axis limits in a misleading way.

2.2.5 Ticks and Tick Labels

Ticks are the marks/values shown along an axis.

You can specify them:

```
plt.xticks([0, 1, 2, 3])
```

You can replace them with labels:

```
plt.xticks(
    [0, 1, 2, 3],
    ["Jan", "Feb", "Mar", "Apr"]
)
```

You can rotate labels:

```
plt.xticks(rotation=45)
```

This is particularly useful when category names are long.

2.2.6 Colors

You can specify colors:

```
plt.plot(months, sales, color="blue")
```

Common colors:

```
"red"
"blue"
"green"
"orange"
"purple"
"black"
```

You can also use hexadecimal colors:

```
color="#2E86AB"
```

Example:

```
plt.bar(
    products,
    sales,
    color="#3498DB"
)
```

2.2.7 Line Styles

Common line styles:

```
-   solid
--  dashed
:   dotted
-.  dash-dot
```

Example:

```
plt.plot(
    months,
    sales,
    linestyle="--"
)
```

You can combine color and line style:

```
plt.plot(
    months,
    sales,
    color="blue",
    linestyle="--"
)
```

2.2.8 Markers

Markers show individual observations.

Examples:

```
plt.plot(
    months,
    sales,
    marker="o"
)
```

Common markers include:

```
o   circle
s   square
^   triangle
*   star
D   diamond
```

Example:

```
plt.plot(
    months,
    sales,
    marker="o",
    markersize=8
)
```

2.2.9 Transparency — `alpha`

`alpha` controls transparency.

Range:

```
0 → completely transparent
1 → completely opaque
```

Example:

```
plt.scatter(
    x,
    y,
    alpha=0.5
)
```

This is especially useful for scatter plots where many observations overlap.

2.2.10 Gridlines

Add a grid:

```
plt.grid()
```

Or:

```
plt.grid(
    linestyle="--",
    alpha=0.5
)
```

Gridlines can make numerical values easier to read.

Don't overuse them. A very heavy grid can make the chart visually noisy.

2.2.11 Legends

When a chart contains multiple data series, use a legend.

```
plt.plot(months, sales_2024, label="2024")
plt.plot(months, sales_2025, label="2025")

plt.legend()
```

Without the legend, the viewer doesn't know which line represents which dataset.

2.2.12 Fonts and Text

You can customize text:

```
plt.title(
    "Monthly Sales",
    fontsize=18,
    fontweight="bold"
)
```

Axis labels:

```
plt.xlabel(
    "Month",
    fontsize=12
)
```

You can also add text directly to the chart:

```
plt.text(
    2,
    180,
    "Highest Sales"
)
```

Module 3 — Essential Matplotlib Charts

3.1 Line Charts

What is a line chart?

A line chart connects data points with lines.

It's particularly useful for:

- trends
- time series
- changes over an ordered variable

3.1.1 Basic Line Chart

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [100, 120, 150, 140, 180]

plt.plot(months, sales)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```

3.1.2 Multiple Lines

```
sales_2024 = [100, 120, 150, 140, 180]
sales_2025 = [120, 135, 160, 155, 200]

plt.plot(
    months,
    sales_2024,
    label="2024"
)

plt.plot(
    months,
    sales_2025,
    label="2025"
)

plt.legend()
plt.show()
```

Now the chart can answer:

> How did sales change between the two years?

3.1.3 Markers

```
plt.plot(
    months,
    sales,
    marker="o"
)
```

More customization:

```
plt.plot(
    months,
    sales,
    marker="o",
    markersize=7,
    linewidth=2
)
```

3.1.4 Line Styles

```
plt.plot(
    months,
    sales_2024,
    linestyle="-",
    label="2024"
)

plt.plot(
    months,
    sales_2025,
    linestyle="--",
    label="2025"
)
```

3.1.5 Comparing Trends

A line chart is useful when asking:

- Is revenue increasing?
- Which product is growing faster?
- Which month had the highest sales?
- Are two trends moving together?

Example:

```
plt.plot(months, product_a, label="Product A")
plt.plot(months, product_b, label="Product B")

plt.legend()
plt.grid(alpha=0.3)

plt.show()
```

3.1.6 Customizing Axes

```
plt.ylim(0, 250)

plt.xlabel("Month")
```

```
plt.ylabel("Revenue")

plt.xticks(rotation=45)
```

3.1.7 Adding Annotations

Annotations highlight important observations.

```
plt.plot(months, sales, marker="o")

plt.annotate(
    "Peak sales",
    xy=("May", 180),
    xytext=("Mar", 200),
    arrowprops=dict(arrowstyle="->")
)

plt.show()
```

This is useful for emphasizing:

- peaks
- drops
- unusual values
- important business events

3.2 Bar Charts

What is a bar chart?

Bar charts are mainly used to **compare categories**.

Example:

```
Product A ████████████
Product B ████████
Product C ██████
```

3.2.1 Vertical Bar Chart

```
products = ["Laptop", "Phone", "Tablet"]
sales = [500, 700, 300]

plt.bar(products, sales)

plt.title("Sales by Product")
plt.xlabel("Product")
plt.ylabel("Sales")

plt.show()
```

3.2.2 Horizontal Bar Chart

Use `barh()`:

```
plt.barh(products, sales)

plt.title("Sales by Product")
plt.xlabel("Sales")
plt.ylabel("Product")

plt.show()
```

Horizontal bars are particularly useful when category names are long.

3.2.3 Multiple Bars

Suppose we want to compare two years:

```
products = ["Laptop", "Phone", "Tablet"]

sales_2024 = [500, 700, 300]
sales_2025 = [600, 800, 450]
```

We need positions for the bars:

```
import numpy as np

x = np.arange(len(products))
width = 0.35

plt.bar(
    x - width/2,
    sales_2024,
    width,
    label="2024"
)

plt.bar(
    x + width/2,
    sales_2025,
    width,
    label="2025"
)

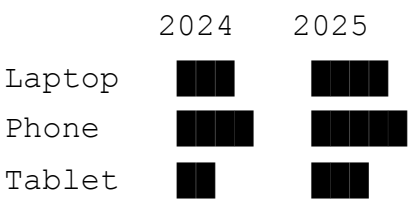
plt.xticks(x, products)
plt.legend()

plt.show()
```

3.2.4 Grouped Bar Chart

A grouped bar chart puts related bars next to each other.

Example:



Useful for:

> Comparing multiple groups across categories.

3.2.5 Stacked Bar Chart

Instead of placing bars beside each other, stack them.

```
plt.bar(
    products,
    online_sales,
    label="Online"
)

plt.bar(
    products,
    store_sales,
    bottom=online_sales,
    label="Store"
)

plt.legend()
plt.show()
```

This allows you to see:

- total value
- composition of that total

3.2.6 Data Labels

Data labels show the exact value above each bar.

```
bars = plt.bar(products, sales)

for bar in bars:
    value = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width()/2,
        value,
        str(value),
        ha="center",
```

```
        va="bottom"
    )

plt.show()
```

This is useful when exact values matter.

3.2.7 Custom Colors

```
colors = ["#3498DB", "#E74C3C", "#2ECC71"]

plt.bar(
    products,
    sales,
    color=colors
)

plt.show()
```

3.3 Histograms

What is a histogram?

A histogram shows the **distribution of numerical data**.

Unlike a bar chart, a histogram groups continuous numerical values into ranges called **bins**.

Example:

```
Frequency
  |
  |       ███
  |     ██████
  |   ██████████
  | ██████████████
  +--------------------> Age
    10 20 30 40 50 60
```

3.3.1 Creating a Histogram

```
ages = [18, 21, 22, 25, 26, 28, 30, 32, 35, 35, 40]

plt.hist(ages)

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
```

3.3.2 Understanding Bins

A **bin** is an interval into which observations are grouped.

For example:

```
20–29
30–39
40–49
50–59
```

You can specify the number of bins:

```
plt.hist(
    ages,
    bins=5
)
```

Or specify exact boundaries:

```
plt.hist(
    ages,
    bins=[0, 10, 20, 30, 40, 50, 60]
)
```

3.3.3 Changing Bin Size

More bins:

```
plt.hist(data, bins=20)
```

Fewer bins:

```
plt.hist(data, bins=5)
```

Important teaching point

Too few bins can hide patterns.

Too many bins can make the distribution noisy.

The bin choice affects how the distribution appears.

3.3.4 Density

Instead of showing frequency, you can show **probability density**:

```
plt.hist(
    ages,
    bins=10,
    density=True
)
```

The total area of the histogram represents 1.

This is useful when comparing distributions with different sample sizes.

3.3.5 Multiple Distributions

You can compare two distributions:

```
plt.hist(
    group_a,
    bins=10,
    alpha=0.5,
    label="Group A"
)
```

```
plt.hist(
    group_b,
    bins=10,
    alpha=0.5,
    label="Group B"
)

plt.legend()
plt.show()
```

`alpha` makes overlapping distributions easier to see.

3.3.6 Interpreting Distributions

Teach students to look for:

- Center
- Spread
- Shape
- Skewness
- Peaks
- Gaps
- Possible outliers

Ask:

> Is the distribution symmetric?

> Is it right-skewed?

> Where are most observations concentrated?

> Are there unusual observations?

3.4 Scatter Plots

What is a scatter plot?

A scatter plot shows the relationship between **two numerical variables**.

Example:

> Experience vs Salary

```
experience = [1, 2, 3, 4, 5, 6]
salary = [30, 35, 38, 45, 52, 60]

plt.scatter(experience, salary)

plt.xlabel("Experience")
plt.ylabel("Salary")

plt.show()
```

3.4.1 Color

```
plt.scatter(
    experience,
    salary,
    color="blue"
)
```

3.4.2 Size

Use `s`:

```
plt.scatter(
    experience,
    salary,
    s=100
)
```

You can also make size depend on another variable:

```
plt.scatter(
    experience,
    salary,
```

```
    s=company_size
)
```

3.4.3 Transparency

```
plt.scatter(
    experience,
    salary,
    alpha=0.5
)
```

Useful when many points overlap.

3.4.4 Multiple Groups

```
plt.scatter(
    experience_it,
    salary_it,
    label="IT"
)

plt.scatter(
    experience_sales,
    salary_sales,
    label="Sales"
)

plt.legend()
plt.show()
```

3.4.5 Trend Interpretation

Students should understand:

Positive relationship

As X increases, Y tends to increase.

Negative relationship

As X increases, Y tends to decrease.

No obvious relationship

The points don't show a clear pattern.

Important:

> **Correlation does not automatically mean causation.**

A scatter plot can show an association, but it doesn't prove that one variable causes the other.

3.5 Pie Charts

What is a pie chart?

A pie chart shows how categories contribute to a **whole**.

Example:

```
labels = ["Laptop", "Phone", "Tablet"]
sales = [40, 35, 25]

plt.pie(
    sales,
    labels=labels
)

plt.show()
```

3.5.1 Labels

```
plt.pie(
    sales,
```

```
    labels=labels
)
```

3.5.2 Percentages

Use `autopct`:

```
plt.pie(
    sales,
    labels=labels,
    autopct="%1.1f%%"
)
```

3.5.3 Exploding Slices

You can highlight a category:

```
explode = [0.1, 0, 0]

plt.pie(
    sales,
    labels=labels,
    explode=explode,
    autopct="%1.1f%%"
)
```

3.5.4 Donut Charts

A donut chart is essentially a pie chart with a hole in the middle.

```
plt.pie(
    sales,
    labels=labels,
    autopct="%1.1f%%",
    wedgeprops={"width": 0.4}
)
```

```
plt.show()
```

Practical advice

Teach students that pie/donut charts work best with **a small number of categories** and a meaningful whole.

If precise comparisons are important, a bar chart is often easier to read.

3.6 Box Plots

What is a box plot?

A box plot summarizes the distribution of numerical data.

It helps identify:

- Median
- Quartiles
- Spread
- Potential outliers

3.6.1 Creating a Box Plot

```
salaries = [30, 35, 40, 42, 45, 48, 50, 55, 90]

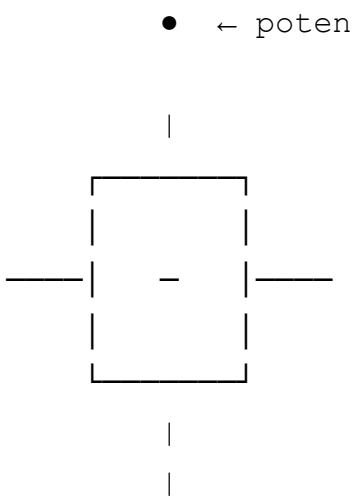
plt.boxplot(salaries)

plt.ylabel("Salary")

plt.show()
```

3.6.2 Understanding the Box

Conceptually:



The important concepts are:

Median

The line inside the box.

Q1

25th percentile.

Q3

75th percentile.

IQR

Interquartile range:

IQR = Q3 – Q1

Whiskers

Extend from the box to values within the standard whisker range.

Outliers

Observations beyond the whisker limits may be displayed separately.

3.6.3 Comparing Multiple Groups

```
salary_it = [...]
salary_sales = [...]
salary_hr = [...]

plt.boxplot(
    [salary_it, salary_sales, salary_hr],
    labels=["IT", "Sales", "HR"]
)

plt.ylabel("Salary")

plt.show()
```

This makes it possible to compare the distributions between departments.

Module 4 — Advanced Matplotlib Customization

This module should teach students how to move from:

> **"I can create a chart."**

to:

> **"I can create a clear, professional visualization."**

4.1 `figsize`

```
fig, ax = plt.subplots(
    figsize=(10, 6)
)
```

Example:

```
fig, ax = plt.subplots(figsize=(12, 7))

ax.bar(products, sales)

plt.show()
```

4.2 DPI

DPI means **dots per inch**.

It controls image resolution.

For example:

```
plt.figure(
    figsize=(10, 6),
    dpi=100
)
```

When exporting:

```
plt.savefig(
    "sales.png",
    dpi=300
)
```

Higher DPI generally gives a higher-resolution raster image, though it also increases file size.

4.3 Colors and Color Palettes

Instead of using random colors, choose colors intentionally.

Example:

```
colors = [
    "#3498DB",
    "#E74C3C",
    "#2ECC71"
]
```

```
plt.bar(
    products,
    sales,
    color=colors
)
```

Teaching principle

Use color to communicate something.

For example:

- One color → standard categories
- Different colors → meaningful groups
- Highlight color → important observation

Avoid using many bright colors without purpose.

4.4 Transparency

```
plt.scatter(
    x,
    y,
    alpha=0.5
)
```

Or:

```
plt.plot(
    x,
    y,
    alpha=0.7
)
```

Useful for:

- overlapping points
- multiple distributions
- reducing visual intensity

4.5 Fonts

```
plt.title(
    "Sales Analysis",
    fontsize=20,
    fontweight="bold"
)

plt.xlabel(
    "Product",
    fontsize=12
)

plt.ylabel(
    "Sales",
    fontsize=12
)
```

You can also set font family:

```
plt.rcParams["font.family"] = "DejaVu Sans"
```

4.6 Font Sizes

A simple hierarchy is useful:

```
Title       → largest
Subtitle    → medium
Axis labels → smaller
Tick labels → smaller
Annotations → selective
```

This makes the chart easier to scan.

4.7 Line Widths

```
plt.plot(
    months,
    sales,
    linewidth=3
)
```

Use thicker lines for emphasis.

4.8 Markers

```
plt.plot(
    months,
    sales,
    marker="o",
    markersize=8
)
```

You can customize marker appearance:

```
plt.plot(
    months,
    sales,
    marker="o",
    markerfacecolor="white",
    markeredgecolor="blue"
)
```

4.9 Grid Customization

```
plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)
```

You don't necessarily need a grid on both axes.

For many business charts, a subtle horizontal grid is enough to help compare values.

4.10 Tick Customization

Example:

```
plt.xticks(
    rotation=45,
    fontsize=10
)

plt.yticks(
    fontsize=10
)
```

You can also control which values appear:

```
plt.yticks([0, 100, 200, 300, 400])
```

4.11 Axis Limits

```
plt.xlim(0, 10)
plt.ylim(0, 500)
```

Object-oriented:

```
ax.set_xlim(0, 10)
ax.set_ylim(0, 500)
```

Important principle

Axis limits should support interpretation rather than distort it.

For bar charts in particular, students should be cautious about truncating the baseline when comparing bar lengths.

4.12 Spines

Spines are the lines around the plotting area.

You can access them through the Axes object:

```
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
```

Example:

```
fig, ax = plt.subplots()

ax.bar(products, sales)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

plt.show()
```

This can create a cleaner appearance.

4.13 Legends

```
ax.legend(
    loc="upper left"
)
```

You can customize:

```
ax.legend(
    title="Year",
    fontsize=10
)
```

A legend should explain something that isn't already obvious.

4.14 Annotations

Annotations are used to call attention to specific points.

```
ax.annotate(
    "Highest sales",
    xy=("May", 200),
```

```
    xytext=("Mar", 230),
    arrowprops=dict(
        arrowstyle="->"
    )
)
```

Good uses:

- highest value
- lowest value
- unusual spike
- important event
- target achievement

4.15 Text on Charts

You can place text directly:

```
ax.text(
    2,
    150,
    "Important point"
)
```

You can customize alignment:

```
ax.text(
    2,
    150,
    "Peak",
    ha="center",
    va="bottom"
)
```

4.16 Subplots

Subplots allow multiple charts within one Figure.

```
fig, axes = plt.subplots(
    2,
```

```
    2,
    figsize=(12, 8)
)
```

This creates:

```
┌──────────────┬──────────────┐
│    Chart 1   │   Chart 2    │
├──────────────┼──────────────┤
│    Chart 3   │   Chart 4    │
└──────────────┴──────────────┘
```

4.16.1 Multiple Rows and Columns

Example:

```
fig, axes = plt.subplots(
    2,
    2,
    figsize=(12, 8)
)

axes[0, 0].plot(months, sales)
axes[0, 1].bar(products, product_sales)
axes[1, 0].hist(ages)
axes[1, 1].scatter(experience, salary)

plt.show()
```

4.17 Sharing Axes

Sometimes multiple charts should use the same axis scale.

```
fig, axes = plt.subplots(
    2,
    1,
    sharex=True
)
```

This makes comparison easier.

For example:

```
fig, axes = plt.subplots(
    2,
    1,
    sharex=True,
    figsize=(10, 8)
)

axes[0].plot(months, sales)
axes[1].plot(months, profit)

plt.show()
```

Both charts use the same x-axis.

4.18 Figure-Level vs Axes-Level Customization

This is an important concept for students.

Figure-level

Applies to the entire Figure.

Examples:

```
fig.suptitle("Sales Analysis")
fig.set_size_inches(12, 8)
```

Axes-level

Applies to a particular plot.

Examples:

```
ax.set_title("Monthly Sales")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue")
```

Think:

```
Figure
 ├── Axes 1
 ├── Axes 2
 ├── Axes 3
 └── Axes 4
```

Figure-level settings affect the whole canvas.

Axes-level settings affect a particular plotting area.

4.19 Combining Different Chart Types

You can combine charts when it improves communication.

For example, a bar chart with a line showing a target:

```
fig, ax = plt.subplots(figsize=(10, 6))

ax.bar(months, sales, label="Sales")

ax.plot(
    months,
    target,
    color="red",
    marker="o",
    label="Target"
)

ax.set_title("Sales vs Target")
ax.set_ylabel("Revenue")

ax.legend()

plt.show()
```

This can answer:

> Which months exceeded the target?

Use combinations carefully. The goal is clarity, not visual complexity.

4.20 `tight_layout()`

When you have multiple charts or long labels, use:

```
plt.tight_layout()
```

It helps prevent elements from overlapping.

Example:

```
fig, axes = plt.subplots(2, 2)

# charts...

plt.tight_layout()
plt.show()
```

For more complex figures, you may also encounter:

```
fig, axes = plt.subplots(
    2,
    2,
    constrained_layout=True
)
```

4.21 Exporting Charts

A chart isn't always just displayed on screen. Analysts often need to put charts into:

- PowerPoint
- Reports
- PDFs
- Documents
- Websites
- Presentations

Use:

```
plt.savefig("sales.png")
```

PNG

```
plt.savefig(
    "sales.png",
    dpi=300
)
```

Good for general raster images.

JPEG

```
plt.savefig(
    "sales.jpg",
    dpi=300
)
```

Useful for photographic-style raster output, though PNG is often preferable for charts.

SVG

```
plt.savefig("sales.svg")
```

SVG is a **vector format**, which is useful when you want graphics to remain sharp when resized.

PDF

```
plt.savefig("sales.pdf")
```

Useful for reports and documents.

4.22 Resolution — DPI

For raster formats such as PNG/JPEG:

```
plt.savefig(
    "chart.png",
```

```
    dpi=300
)
```

Common idea:

- Lower DPI → smaller file / lower resolution
- Higher DPI → larger file / higher resolution

For reports and presentations, 150–300 DPI is often sufficient depending on the use case.

4.23 Bounding Box

You may see:

```
plt.savefig(
    "chart.png",
    bbox_inches="tight"
)
```

`bbox_inches="tight"` tells Matplotlib to reduce unnecessary surrounding whitespace.

Example:

```
plt.savefig(
    "sales.png",
    dpi=300,
    bbox_inches="tight"
)
```

Complete Professional Example

Here's a good example to finish these modules with:

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [120, 150, 135, 180, 200, 220]
target = [130, 140, 150, 160, 180, 200]

fig, ax = plt.subplots(figsize=(10, 6))
```

```
# Sales
ax.plot(
    months,
    sales,
    marker="o",
    linewidth=2.5,
    color="#2E86AB",
    label="Actual Sales"
)

# Target
ax.plot(
    months,
    target,
    linestyle="--",
    linewidth=2,
    color="#E74C3C",
    label="Target"
)

# Title and labels
ax.set_title(
    "Monthly Sales vs Target",
    fontsize=18,
    fontweight="bold"
)

ax.set_xlabel("Month", fontsize=12)
ax.set_ylabel("Sales", fontsize=12)

# Grid
ax.grid(
    axis="y",
    linestyle="--",
    alpha=0.3
)

# Remove unnecessary spines
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
```

```
# Legend
ax.legend()

# Highlight June
ax.annotate(
    "Highest sales",
    xy=("Jun", 220),
    xytext=("Apr", 230),
    arrowprops=dict(
        arrowstyle="->"
    )
)

# Layout
plt.tight_layout()

# Save
plt.savefig(
    "monthly_sales.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

This single example brings together:

- Figure
- Axes
- Line chart
- Multiple lines
- Markers
- Line styles
- Colors
- Title
- Axis labels
- Grid
- Spines
- Legend
- Annotation
- Figure size

- DPI
- Exporting
- `tight_layout()`

Suggested Practice Exercises

To make these modules genuinely useful for beginners, I'd attach exercises to each section.

Exercise 1 — Basic Line Chart

Given:

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]
revenue = [100, 120, 150, 130, 180]
```

Create a properly labelled line chart.

Exercise 2 — Product Comparison

Given:

```
products = ["Laptop", "Phone", "Tablet", "Monitor"]
sales = [500, 800, 300, 450]
```

Create:

- Vertical bar chart
- Horizontal bar chart
- Add data labels
- Use custom colors

Exercise 3 — Distribution

Given customer ages, create a histogram.

Experiment with:

```
bins=5
bins=10
bins=20
```

Ask:

> How does changing the bin size change the appearance of the distribution?

Exercise 4 — Relationship

Given:

```
Years of Experience
Salary
```

Create a scatter plot.

Then ask:

> Does there appear to be a positive, negative, or weak relationship?

Exercise 5 — Group Comparison

Create box plots for:

```
IT salaries
Sales salaries
HR salaries
```

Ask:

- Which group has the highest median?
- Which group has the greatest spread?
- Are there potential outliers?

Exercise 6 — Professional Visualization

Give students a raw dataset and require them to create **three charts**, with the following rule:

> Every chart must have a meaningful title, axis labels, appropriate colors, readable ticks, and at least one useful analytical observation.

That last requirement is important. It shifts the course from **"learning Matplotlib commands"** toward **"using visualization for data analysis."**

The core takeaway for students

I'd end these modules with this mental model:

